mindX Knowledge Catalogue — A Subsystem Specification

The mindX knowledge catalogue is best built as a **CQRS projection layer over the existing append-only memory log**, not as a parallel store. Operational logs remain the sole source of truth; the catalogue is a set of versioned, idempotent read-models — a graph index, a vector index, a keyword index, and a relational document store — derived from those logs by named projector services. This preserves the "logs are memories" axiom exactly: nothing in the catalogue is canonical, everything in the catalogue can be rebuilt by replaying the log. The model is borrowed directly from DataHub's MetadataChangeProposal/MetadataChangeLog architecture, formalized by Greg Young's CQRS and validated for AI agents in 2025–2026 work (DPM, ESAA, PROV-AGENT, Graphiti's episode subgraph). The metadata schema itself adopts Google Cloud Dataplex Universal Catalog's six-resource model (EntryGroup, EntryType, AspectType, Entry, EntryLink, EntryLinkType) — the most expressive and battle-tested metadata schema in production today — re-expressed in Pydantic v2 and rendered Python-native, Podman-deployable, and free of GCP dependencies. The result is a system in which agent skills, memories, runs, and artifacts become first-class catalogued entities discoverable by hybrid retrieval (BM25 + dense vectors + graph traversal + cross-encoder rerank), with cryptographically verifiable lineage that can later be anchored on-chain by openBDK without entangling the catalogue with chain semantics.

The architectural axiom: catalogue as projection of memory

The mindX log is an append-only, content-addressed Merkle DAG of `AgentEvent` records. Every memory write, tool call, skill invocation, alignment scoring event, and POD state transition is an event. The catalogue is **never** written to directly. Producers append events; **projectors** consume them and update read-models. This is the same pattern Martin Fowler called Event Sourcing, and that DataHub realises at scale on Kafka. It collapses several otherwise-hard problems into one design: rebuild from scratch is `replay-the-log`, audit is `where-clause on log`, time-travel is `read-up-to-offset`, GDPR erasure is handled by storing personal payloads outside the log (referenced by id) and dropping the side table, and **multi-tenant federation is just multiple consumers of the same log.** Martin Fowler

Two recent academic frames are worth citing because they describe the mindX situation almost verbatim. *Stateless Decision Memory for Enterprise AI Agents* (arXiv 2604.20158) defines **Deterministic Projection Memory**: the trajectory is the immutable event log; at decision time the agent runs a single task-conditioned projection at temperature zero; replay from the same log under the same model version produces the same memory view. *ESAA: Event Sourcing for Autonomous Agents* (arXiv 2602.23193) adopts CQRS for software-engineering agents, with the source of truth being "an immutable log of intentions, decisions, and effects, from which the current state is deterministically projected." mindX is already operating on this thesis; the catalogue is the missing read-side.

The implication for "logs are memories" is precise: **the catalogue does not replace memory; it indexes it**. Every catalogue entry carries a `source_event_cids` field pointing back to the log events that produced it. Burn the catalogue and rebuild it; the memories are untouched. This is the non-violation contract.

The six-resource metadata model

Adopt Dataplex's six-resource decomposition verbatim, because it is the only production metadata schema rich enough to express skills, memories, runs, alignment scores, agents, and POD states as instances of one consistent model:

EntryGroup is a namespace and access-control unit (e.g. `agent:codephreak`, `domain:bankon`, `pod:level-3`). **EntryType** is the schema that an Entry must satisfy — it declares an immutable list of `requiredAspects`. **AspectType** is the recursive schema for an aspect, defined as a Pydantic-or-JSON-Schema model with stable integer field indexes (Protobuf-style) for safe forward/backward evolution. **Entry** is a typed instance with a key plus a `map<aspect_key, Aspect>` of attached aspects (entry-level or column/path-scoped via `aspect_key@path`). **EntryLink** is a typed first-class edge between two entries; built-in link types include `synonym`, `related`, `definition`, `derivedFrom`, `producedBy`, `wasInformedBy`. **EntryLinkType** schemas the edge. google + 5

Aspects are the central extensibility primitive. They classify along three axes: required-vs-optional (declared by the EntryType), system-vs-custom (system aspects are projector-managed, custom aspects user-attachable), and metadata-content category (technical, business, operational, data-derived). Lineage is **not** an aspect — it lives in a parallel graph keyed by FQN, modelled as OpenLineage/PROV-AGENT events, because lineage is execution-derived and aspects are curatorial. Keep them distinct. Google Cloud + 2

URNs are the universal identifier: `urn:mindx:<kind>:<scope>:<name>[:v<semver>]`. Examples: `urn:mindx:skill:research.web:grep:v1.4.0`, `urn:mindx:memory:agent.hands.7:bafyrei…`, `urn:mindx:run:01H8X…` (UUIDv7), `urn:mindx:agent:soul.alpha`. URNs are stable across versions; CIDs (IPLD content identifiers) change with every payload mutation, giving both stable identity and per-version cryptographic immutability.

Component decomposition

mindX's preference for clearly named, composable, swappable modules drives the following package layout. Each component has one responsibility, one interface, one scaling story.

`mindx.catalogue.core` owns the metadata model itself — Pydantic v2 classes for `Entry`, `Aspect`, `EntryType`, `AspectType`, `EntryGroup`, `EntryLink`, `EntryLinkType`, plus an `AspectRegistry` that loads aspect-type definitions from TOML/YAML manifests at boot and supports hot-reload via `watchfiles`. Schema evolution rules are enforced here: stable integer

field indexes never reuse, types never change, fields can be marked `deprecated` but not deleted. Code-generation from JSON Schema (using `datamodel-code-generator`) gives Python clients and TypeScript clients from one source of truth — OpenMetadata's pattern, adapted. **Scaling:** stateless library, embedded in every other component. google Restack

`mindx.catalogue.log` is the thin adapter to mindX's existing append-only memory log. It exposes `append(event)` and `subscribe(subject_pattern, durable_name) -> AsyncIterator[CatalogueEvent]` over NATS JetStream. Each `CatalogueEvent` is a `msgspec.Struct` carrying `{cid, prev_cid, seq, ts, actor, kind, payload, sig}`. The component does not own the log — it is a typed view onto it. This is the only component that touches the log substrate; everything else reads through it. **Scaling:** stateless; multiple consumers fan out; JetStream durable consumers checkpoint per projection.

`mindx.catalogue.ingest` is the projector framework. A projector is a named async consumer with a `version` string; bumping the version triggers a full replay rebuild. Each projector implements `apply(state, event) -> new_state` (pure, idempotent). Projectors are discovered via `pluggy` entry-points in the group `mindx.catalogue.projectors`, so adding a new projection is `pip install mindx-projector-foo`. Built-in projectors: `proj_entries` (writes to Postgres source-of-truth document table), `proj_graph` (writes to Kuzu/Neo4j edges), `proj_search` (writes to Meilisearch), `proj_vector` (chunks → embeds → writes to Qdrant), `proj_lineage` (folds OpenLineage RunEvents into a DAG), `proj_skills` (rolling stats). **Scaling:** horizontal by JetStream consumer-group partitioning (typically by `entity_urn` hash); each projector scales independently to its own backpressure.

`mindx.catalogue.entries` is the document store of the source-of-truth catalogue projection. PostgreSQL with a single `entity_aspect` table mirroring DataHub's `metadata_aspect_v2`: `(entity_urn, aspect_name, version, json_payload, created_at, created_by)`. Versioned aspects keep history, timeseries aspects live in a partitioned companion table. Atomic CRUD on a whole entry is a transaction over the rows for that URN. Optional Apache AGE extension layers a graph view on the same Postgres for small deployments. **Scaling:** vertical first (Postgres scales to billions of rows on a single host with proper partitioning); horizontal via logical sharding by URN namespace once you outgrow a single primary. DataHub +2 Google Cloud

`mindx.catalogue.graph` holds the relationship graph: `EntryLink` edges and entity-level traversals. Default backend is **Kuzu** (MIT, embedded, Cypher, vector + FTS built in) for analytical lineage and curatorial-link queries on every node. When a globally queryable graph is needed across federated instances, swap the backend to Neo4j Community or Apache AGE without changing Cypher. Bi-temporal edges per Graphiti's pattern: every edge carries `t_valid_from`, `t_valid_to`, `t_ingested`; **invalidate, never delete**, so contradicting facts coexist for audit. **Scaling:** vertical via Kuzu's columnar layout (single-node handles tens of millions of edges); horizontal via per-namespace graph instances stitched at query time by `mindx.catalogue.search`. GitHub + 2

`mindx.catalogue.lineage` is the OpenLineage-and-PROV-AGENT layer. Every agent run emits an OpenLineage RunEvent (`START`, `RUNNING`, `COMPLETE`, `FAIL`) to the log; the lineage projector folds these into a Run/Job/Dataset DAG with custom facets for `prompt`, `model`, `tokensIn/Out`, `cost`, `safetyClassification`, `parentRun`, `mindxAlignmentScore`. The OpenLineage `Job` becomes a skill invocation definition; the `Run` is the invocation instance (UUIDv7); the `Dataset` is any first-class artifact (memory chunk, generated document, code patch, embedding, tool result). Mirror selected events to Marquez via `HttpTransport` for free visualisation; that integration is opt-in and removable. PROV-AGENT classes (`AIAgent`, `AgentTool`, `AIModelInvocation`, `Prompt`, `AIModel`, `ResponseData`) are projected into the graph for "why does the agent know X?" / "which prompts produced bad outputs?" queries. **Scaling:** lineage events are append-only and partition naturally by run_id; the projector scales horizontally with no coordination. Openlineage + 4

`mindx.catalogue.search` is the hybrid retrieval engine. It exposes a single `search(query, kinds, modes, filters, k) -> RankedResults` interface and fans out internally to three legs: BM25 over Meilisearch (or SQLite FTS5 in minimal mode), dense k-NN over Qdrant (or sqlite-vec in minimal mode), and graph traversal over Kuzu. Results are fused with reciprocal rank fusion, then cross-encoder reranked via `bge-reranker-v2-m3`. IAM-aware filtering happens at result time, not query time — Dataplex's pattern, which prevents permission leaks via inferred existence. **Scaling:** each leg scales independently; the fuser is stateless and async. Baseten

`mindx.catalogue.skills` is the skill/tool registry, modelled as a specialization of `core` with the `Skill` EntryType. Skill descriptors are MCP-compatible at the wire level (`name`, `description`, `inputSchema`, `outputSchema`, `annotations`) plus mindX extensions: `embedding` (vector of `name+description+IO_schema`), `cost` (token/USD/latency p50/p95), `side_effects`, `permissions`, `preconditions`, `composability` (chain/parallel compatibility), `provenance` (which agent authored, derived from which skill), `stats.success_by_task_pattern` (Cognee-style rolling success rate per task pattern, used for routing). The component exposes the registry as an MCP server endpoint so agents themselves discover skills via `tools/list` and `tools/call`, with `notifications/tools/list_changed` propagating updates from JetStream. This is exactly what DataHub's MCP Server and Anthropic's Tool Search Tool do, applied to mindX. **Scaling:** read-heavy; cache the registry per-agent, invalidate on `list_changed`. Modelcontextprotocol

`mindx.catalogue.memory` is the memory-entry projection — the catalogue's view onto agent memory. Each `MemoryEntry` carries `kind` (episodic/semantic/procedural/working), `scope` (agent/session/org), `content`, `embedding` (vector ref), `graph_refs` (entities and edges extracted), `temporal` (bi-temporal validity), `lineage` (PROV-AGENT slice), and `policy` (`source_authority` ∈ {log, projection, derived, extracted}, `ttl_class`, `permissions`, `consolidation_state`). mem0's two-phase extract-update pipeline runs as a projector: extract atomic facts via LLM, then ADD/UPDATE/DELETE/NOOP each candidate against the existing graph. Letta's core/recall/archival tier names map to mindX's hot/warm/cold storage. The bi-

temporal Graphiti invariant — invalidate, never delete — applies here too, so the catalogue can answer "what did the agent believe at time T?". **Scaling:** the memory projection is the largest read-model; partition by `agent_id` for horizontal scale. Dwarves Memo + 3

`mindx.catalogue.federation` handles cross-instance synchronization for the AgenticPlace ↔ mindX ↔ BANKON triangle (and any future mindX nodes). Three layers: **(1) NATS JetStream leaf-node mirrors** as the transport — each node runs JetStream, peers via leaf-node connections, and stream mirrors replicate selected subjects across regions. **(2) `pycrdt` (Yjs/Yrs Rust bindings, MIT, actively maintained 2024–2026)** for eventually-consistent shared documents like the federated aspect-type registry, glossary, or tag taxonomy — any node can edit, conflicts merge automatically. **(3) IPFS/IPLD via Kubo HTTP** for content-addressed catalogue snapshots — periodic Merkle roots pinned across nodes, allowing offline replay and providing the optional anchor point for openBDK chain integration. Federation is **partition-tolerant by default**: a node can operate offline and reconcile when it rejoins. **Scaling:** infinite horizontal — each new node is one more JetStream peer; no central coordinator. Yjs

`mindx.catalogue.policy` owns retention, TTL, archival, and access control. Policies are themselves catalogue entries (recursive metadata layering). A policy entry carries a CEL-style predicate over `(entity_kind, scope, source_authority, age, tags)` and an action (`evict`, `archive`, `summarize`, `freeze`). The component subscribes to a heartbeat tick from JetStream and applies policies to the read-models — never to the log. GDPR right-to-be-forgotten is handled per the Microsoft Azure Architecture Center pattern: personal data lives in a side table referenced by id; events keep the id, the side table is dropped, projections rebuild without the personal payload. **Scaling:** policy evaluation is per-entity and embarrassingly parallel.

`mindx.catalogue.api` is the public surface on `mindx.pythai.net`. **FastAPI** primary (Pydantic v2 native, OpenAPI auto-gen) with a minimal Litestar internal projection-side service for high-throughput msgspec-native paths. Endpoints: REST for CRUD, GraphQL for nested traversal, **Server-Sent Events** for reactive subscriptions on entry-aspect changes (`/aspects/{urn}/stream`), and **MCP/JSON-RPC over Streamable HTTP** so AI agents (mindX itself, AgenticPlace agents, BANKON agents) consume the catalogue as a tool. Authentication via JWT capability tokens; permissions match the per-aspect-type IAM model from Dataplex (`aspect.use:<aspect_type_id>`). **Scaling:** stateless behind a load balancer; horizontal pod replicas. google

`mindx.catalogue.openbdk_bridge` is a deliberately-thin opt-in module — the only place where chainmapping concerns are allowed to surface. It exposes `anchor_snapshot(epoch) -> tx_hash` (publishes the epoch's IPLD root CID to whatever chain openBDK maps), `verify_anchor(cid, tx_hash) -> bool`, and `register_skill_chainmap(skill_urn, chain_id)`. The catalogue does not require this bridge; chain anchoring is opt-in per epoch. The bridge knows nothing about catalogue internals — it consumes only `(cid, epoch)` tuples and chain identifiers. This keeps the catalogue / openBDK / x402 / DAIO / Foundry / Algopy concerns unentangled.

Schemas

The three schemas the rest of mindX needs immediately:

Memory entry (`MemoryEntry` aspect set):

```yaml
```

```yaml
key:
  urn: urn:mindx:memory:<scope>:<id>
  cid: <IPLD CID of latest version>
aspects:
  identity:                          # required
    kind: episodic|semantic|procedural|working
    scope: { agent_id, session_id?, org_id? }
  content:                           # required
    text: str
    structured: object?              # mem0-style extracted facts
  embedding:                         # optional, attached by proj_vector
    model: bge-m3|nomic-embed-text-v2
    dim: int
    vector_ref: qdrant://collection/point_id
  graph_refs:                        # optional, attached by proj_kg
    entities: [GraphNodeRef]
    edges:    [GraphEdgeRef]
  temporal:                          # required (bi-temporal)
    t_valid_from: ts
    t_valid_to:   ts | null
    t_ingested:   ts
  lineage:                           # required (PROV-AGENT slice)
    was_generated_by: ActivityRef
    used:             [EntityRef]    # prompts, prior memories
    was_attributed_to: AgentRef
    model:            { id, version, temperature }
    prompt_cid:       str
    confidence:       float
  policy:                            # required
    source_authority: log|projection|derived|extracted
    ttl_class:        ephemeral|session|persistent|permanent
    permissions:      [str]
    consolidation_state: raw|extracted|merged|superseded
  alignment:                         # optional, mindX-specific
    score: float
    scorer: AgentRef
    rubric_version: str
  supersedes:    [MemoryRef]?        # invalidate-don't-delete chain
  superseded_by: MemoryRef?
links:
  - { type: derivedFrom, target: <prior memory urn> }
  - { type: producedBy,  target: <run urn> }
```

Skill entry (`SkillEntry`):

```yaml
```

```
key:
  urn: urn:mindx:skill:<namespace>:<name>:v<semver>
  cid: <content hash of full descriptor>
aspects:
  mcp_core:                          # required, MCP-compatible
    name: str (≤128, [A-Za-z0-9_.-])
    description: str                 # primary signal for selection — embedded
    input_schema:  JSONSchema
    output_schema: JSONSchema?
    annotations:
      audience: [user, assistant]
      read_only_hint: bool
      destructive_hint: bool
      idempotent_hint: bool
  embedding:                         # required, semantic discovery
    model: str
    vector_ref: qdrant://...
  capabilities:                      # required
    domain: str                      # filesystem|web|memory|chain|alignment
    tags:   [str]
    side_effects: read_only|idempotent|mutating|external_state
    permissions:  [str]
    preconditions:  [Predicate]?
    postconditions: [Predicate]?
  cost:                              # required (timeseries projection)
    tokens_p50/p95: int
    latency_ms_p50/p95: int
    usd_p50: float
  composability:
    chains_with:    [SkillRef]       # output→input compatible
    parallelizable: bool
  provenance:                        # PROV-O
    was_attributed_to: AgentRef
    was_derived_from:  [SkillRef]?
    source_repo:       URI
  stats:                             # timeseries, written by proj_skills
    invocations: int
    success_rate: float
    success_by_task_pattern: { pattern_id: rate }
    last_used_ts: ts
  endpoint:
    transport: in_process|mcp_stdio|mcp_http|a2a|http
    uri: URI
```

```
  retention:
    deprecated: bool
    successor:  SkillRef?
  chainmap:                          # optional, openBDK bridge metadata
    chain_id: str?
    contract_address: str?
```

Lineage event (the on-the-wire log primitive — OpenLineage-compatible + PROV-AGENT-mappable):

```yaml
LineageEvent:
  cid: str
  event_type: START|RUNNING|COMPLETE|FAIL|ABORT
  event_time: ts
  run:
    run_id: UUIDv7
    parent_run_id: UUIDv7?
    facets:
      agent: { id, role, version }      # PROV-AGENT
      model: { id, provider, version, temperature }
      prompt: { cid, role_template }
      mcp_tool: { name, version, server }
      mindx_alignment: { score, scorer, rubric_version }
  job:
    namespace: str                       # e.g. "mindx.agent.researcher"
    name: str                            # skill or projection name
    facets: { source_code_cid, sql, ... }
  inputs:                                # PROV used
    - { namespace, name, facets: { schema, version_cid } }
  outputs:                               # PROV generated
    - { namespace, name, facets: { schema, statistics, version_cid } }
  prov:
    was_informed_by: [run_id]
    was_attributed_to: agent_id
```

A single event payload satisfies three contracts at once: source-of-truth log entry, OpenLineage emitter payload (Marquez/DataHub/Atlan all consume it as-is), and PROV-AGENT triples after a trivial projection. This means **mindX gets ecosystem interoperability for free** — any OpenLineage backend can replay agent activity and visualise the agent DAG without bespoke tooling.

Hybrid retrieval

The 2026 industry consensus on RAG-style retrieval is that no single signal wins. mindX adopts a four-leg hybrid identical in shape to DataHub semantic search and OpenMetadata's RDF + ES + k-NN trio, with one extra leg:

```
query
  |
  ├──► embed(query, bge-m3)    ──► qdrant.knn(k=50)          ┐
  ├──► tokenize/BM25           ──► meilisearch.search(k=50) ─┤
  ├──► entity_extract(query)   ──► kuzu.subgraph(2-hop)      ─┤
  ├──► community_router        ──► graphrag.community_summary─┤  (optional, sense-making)
  |                                                          |
  |                  fuse (RRF, weights tuned per kind)      |
  |                              |                           |
  ▼                              ▼                           ▼
 metadata-filter (scope, permissions, ttl, source_authority)
                                |
                                ▼
             cross-encoder rerank (bge-reranker-v2-m3)
                                |
                                ▼
             top-k → context-budget packer (Matryoshka truncation)
```

For sense-making queries ("summarise what we learned this week"), an optional GraphRAG-style community-summary projector produces Leiden-clustered summary nodes; the router picks the appropriate community level. For most agent queries (precise lookup), the four-leg fan-out is sufficient. **Anthropic's Tool Search Tool pattern** — the agent queries the registry programmatically rather than receiving the whole tool list in the prompt — applies here verbatim: mindX agents call `mindx.catalogue.search.search(...)` as an MCP tool, reducing context pollution by up to 85%.

Horizontal scaling

The catalogue scales horizontally on three independent axes. **Stream partitioning:** JetStream subjects partition by `urn_namespace` hash; each projector consumer-group handles its slice. **Read-model sharding:** Postgres shards by `urn_namespace`, Qdrant uses per-collection multitenancy, Meilisearch uses per-index, Kuzu runs per-namespace instances joined at query time. **Federation:** every mindX/AgenticPlace/BANKON node runs its own complete catalogue stack; nodes peer via NATS leaf-nodes (transport) and pycrdt (shared-state CRDT layer for the aspect-type registry, glossary, and federated tags).

Eventual vs strong consistency is a per-aspect choice. Versioned aspects (`Ownership`, `Permissions`, alignment scores) write through to Postgres synchronously and propagate to indexes asynchronously — strong on the source of truth, eventual on indexes. Timeseries aspects (`stats`, `cost`, telemetry) are eventual everywhere. The pattern matches DataHub's split between `MetadataChangeLog_Versioned_v1` and `_Timeseries_v1`.

Vertical scaling

Hierarchical depth is built into the model in three ways. **EntryGroups nest** through their key path (`entry_groups/agent.codephreak.hands.7`), giving sub-catalogue scopes. **AspectTypes recurse** via `metadataTemplate.typeRef` — Dataplex's pattern, where a record type can reference another record type by id, allowing a single aspect schema to embed sub-aspects to arbitrary depth. **EntryLinks compose** — an entry link can itself carry aspects (e.g., a `derivedFrom` link can carry a `confidence` aspect), and links can chain to form lineage paths up to 20 levels deep (Dataplex's bounded BFS).

Drill-down from coarse to fine works through aspect-key paths: `aspect_key@path` lets any aspect attach not only to an entry but to a JSONPath inside the entry's schema. So a `pii_classification` aspect can be at the entry level (whole memory is CONFIDENTIAL) and also at `@email_address` (that specific column is RESTRICTED). The agent can navigate from "the whole memory" to "this specific extracted entity" by aspect-key suffixing.

Compositional aspects — aspects that reference other aspects — are achieved via URN-typed fields. An `embedding` aspect's `vector_ref` points at a Qdrant point; a `lineage` aspect's `used` field points at a list of memory URNs; a `policy` aspect's `successor` points at the replacement entry. The graph projector materializes these references as edges, giving "metadata-about-metadata" without coupling the document store to the graph store.

The recommended Python stack

Two stacks, because mindX has multiple deployment shapes (edge agents on a laptop vs. federated production nodes).

Minimal stack — single Podman pod, three containers:

- Schema: **Pydantic v2** (catalogue/API), msgspec on hot paths
- Graph: **Kuzu** embedded (no container)

- Vector: **sqlite-vec** + FTS5 embedded (no container)
- Search: **SQLite FTS5** (same DB)
- Event log: **NATS JetStream** (15 MB Go binary, one container)
- Relational/doc: **SQLite** (embedded)
- API: **FastAPI** + uvicorn (one container)
- Embeddings: **Ollama** running `nomic-embed-text-v2` (one container, CPU-OK)
- Projectors: hand-rolled `asyncio` consumers, `pluggy` plugins (in API container)
- Federation: NATS leaf-node + `pycrdt` for shared taxonomy
- Containers: Podman rootless + Quadlet (`mindx.quadlets`)

Three containers, ~500 MB RAM idle, every license MIT/Apache-2/BSD/PostgreSQL.

Full-scale stack — multi-region federation:

- Schema: Pydantic v2 + msgspec
- Graph: Kuzu per-node + Neo4j Community for shared global graph
- Vector: **Qdrant** server per-region + LanceDB embedded for local agent cache
- Search: **Meilisearch** per-region (MIT, hybrid, single Rust binary)
- Event log: **NATS JetStream super-cluster**; Redpanda swap-in if Kafka ecosystem needed
- Relational/doc: **PostgreSQL 16** + pgvector + JSONB + tsvector
- API: FastAPI public, Litestar internal projection services
- Embeddings: **Text Embeddings Inference (TEI)** for single-model GPU prod, **Infinity** for multi-model/multi-modal; default `BAAI/bge-m3`, code `nomic-embed-code`
- Projectors: named async consumers per aspect; **Dagster** only for nightly reconciliation
- Federation: NATS leaf-nodes + JetStream stream mirrors + pycrdt + Kubo IPFS for snapshots
- Plugins: `pluggy` + entry-points group `mindx.catalogue.*`; YAML/TOML manifests with `watchfiles` hot-reload
- Containers: Podman rootless pods per node, Quadlet-managed, `AutoUpdate=registry`

Two libraries deserve specific notes. **Bytewax should not be adopted** — `waxctl` was archived March 2025, the main repo went quiet shortly after, and the project is reportedly being absorbed into Confluent. Hand-rolled `asyncio` projectors are simpler and dependency-light. **Memgraph and SurrealDB carry BSL/MEL licenses** (source-available, not OSI); they're excluded on the open-source preference. **Redis ≥7.4 is RSAL/SSPL** — the catalogue doesn't need Redis at all.

Podman Quadlet deployment

Each mindX node deploys as one rootless Podman pod named `mindx-node`, with each service as a `.container` Quadlet under `~/.config/containers/systemd/mindx/`. Bundle them as a single `mindx.quadlets` for one-shot install via `podman quadlet install mindx.quadlets`. Use `Pod=mindx.pod` to share networking, `*.network` Quadlets for cross-pod federation, `AutoUpdate=registry` plus `podman auto-update.timer` for hands-free updates, and `loginctl enable-linger $USER` so rootless services persist past logout. Quadlet is the strategic unit format; `podman generate systemd` is deprecated in favor of it. For Compose-heavy migrations, `podlet` (Rust tool) converts existing compose files.

Migration and adoption path

The catalogue can be added to a running mindX system without disruption because every catalogue artifact is derivable from the existing log. Phase the rollout in four steps:

Phase 0 — instrumentation only. Deploy `mindx.catalogue.log` and `mindx.catalogue.core` as libraries inside the existing mindX runtime. Existing memory writes and tool calls begin emitting `CatalogueEvent` records to JetStream. No projectors yet; the catalogue is dark. This is purely additive — if the catalogue module crashes, mindX keeps running.

Phase 1 — backfill from history. Run a one-shot replay job that reads the historical log from offset zero, synthesises `CatalogueEvent` records for each memory and tool call, and publishes them to JetStream with their original timestamps preserved in a `replay` facet. Stand up `proj_entries`, `proj_search`, and `proj_vector` to consume the stream. The catalogue now reflects historical state without touching memory.

Phase 2 — graph + lineage + skills. Add `proj_graph`, `proj_lineage`, and `proj_skills`. Begin emitting OpenLineage RunEvents from the agent runtime (every skill invocation = one Run). The skill registry MCP server comes online. Agents start calling `mindx.catalogue.search.search(...)` for skill discovery; the legacy hardcoded skill list is kept as a fallback for two release cycles.

Phase 3 — federation and policy. Stand up `mindx.catalogue.federation` with NATS leaf-node peering across mindX, AgenticPlace, and BANKON nodes. Activate `mindx.catalogue.policy` with TTL/retention rules — note these only act on projections, never on the log. Optionally enable `mindx.catalogue.openbdk_bridge` for periodic IPLD snapshot anchoring; this remains opt-in per epoch.

At every phase the rollback is trivial: stop the projectors, drop the read-model databases, the log is untouched and mindX continues operating on its memory substrate. **This non-disruptive adoptability is the direct payoff of the CQRS/projection architecture** — it would be impossible if the catalogue were canonical state.

Conclusion

The right framing for mindX's knowledge catalogue is not "we need a metadata store" but "we already have a memory substrate; the catalogue is its read-side." Every architectural decision falls out of that: aspects are projections, lineage is a fold over the log, skill discovery is a search over an indexed projection, federation is just multiple consumers of the same events, and chain anchoring is an opt-in observation of projection snapshots that never blocks reads. The Dataplex six-resource model gives the schema; DataHub's CQRS/event-sourcing gives the runtime; OpenLineage and PROV-AGENT give the lineage wire format; mem0, Letta, and Graphiti give the memory-specific semantics; MCP gives the skill protocol; Pydantic v2, NATS JetStream, Postgres, Kuzu, Qdrant, Meilisearch, FastAPI, and Podman Quadlets give the implementation — every piece open-source, Python-primary, modular, and named.

Three insights from the synthesis are worth carrying forward as design constraints. **First, the catalogue is never the source of truth** — that role belongs irrevocably to the log; treating the catalogue as canonical breaks audit, replay, and federation simultaneously. **Second, lineage and links are different kinds of graph** — lineage is execution-derived (events fold into a DAG), curatorial links are agent- or human-authored (synonyms, glossary definitions); conflating them confuses both. **Third, schema evolution must be planned from day one** — stable integer field indexes (Protobuf-style), `deprecated` annotations, and never-reused indexes are the difference between a catalogue you can grow for years and one you have to rewrite at the first model change. mindX's autonomous agents will produce more aspect-type churn than any human-curated catalogue ever has; the schema-evolution discipline is non-negotiable.

The catalogue, built this way, becomes more than an index of mindX's memories — it becomes the substrate by which mindX agents discover their own skills, audit their own reasoning, and federate with their sibling systems on AgenticPlace and BANKON, while leaving the door open for openBDK to anchor cryptographic proofs of state without ever touching the catalogue's read path.